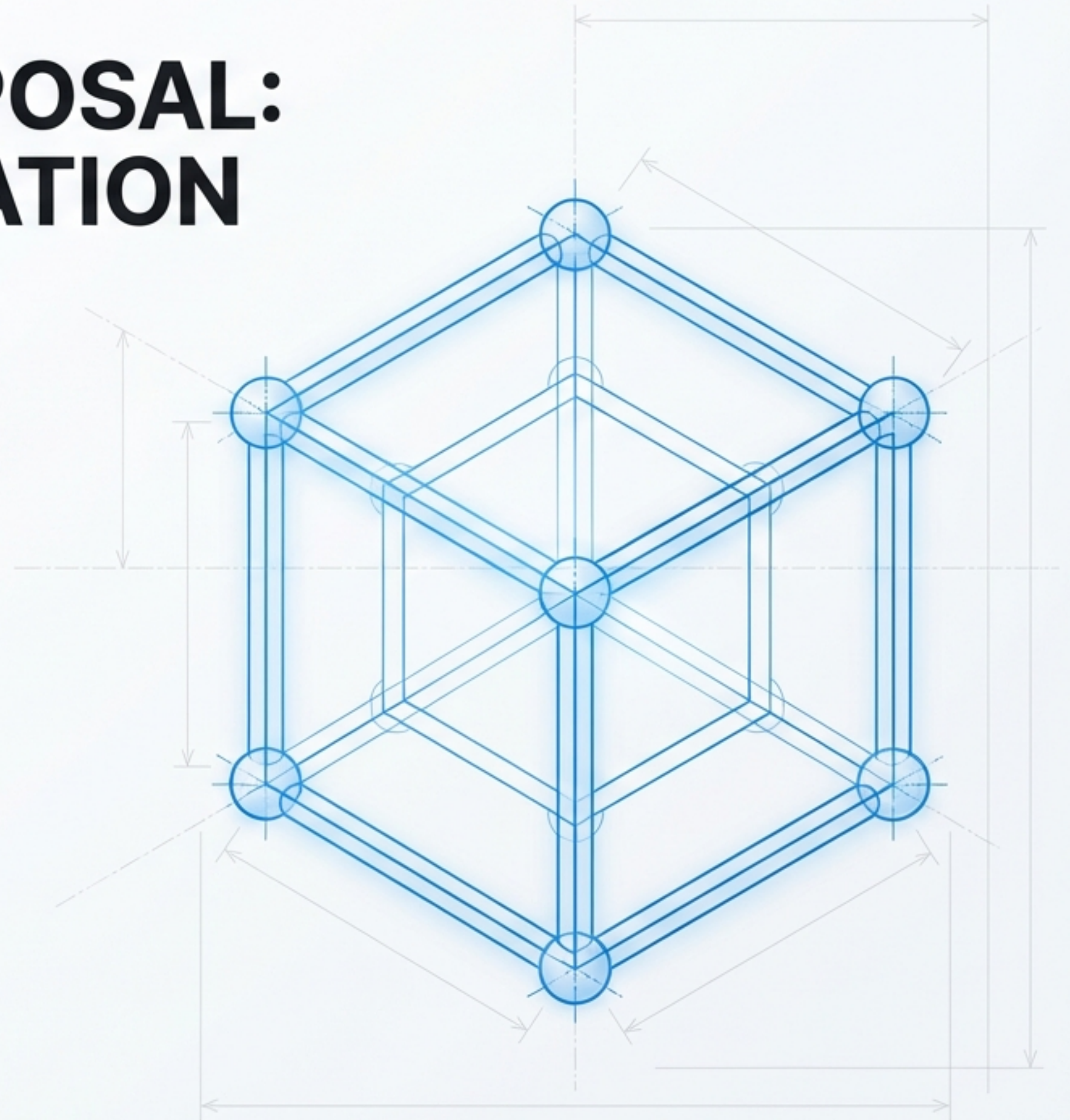


ARCHITECTURE PROPOSAL: MGRAPH-DB INTEGRATION

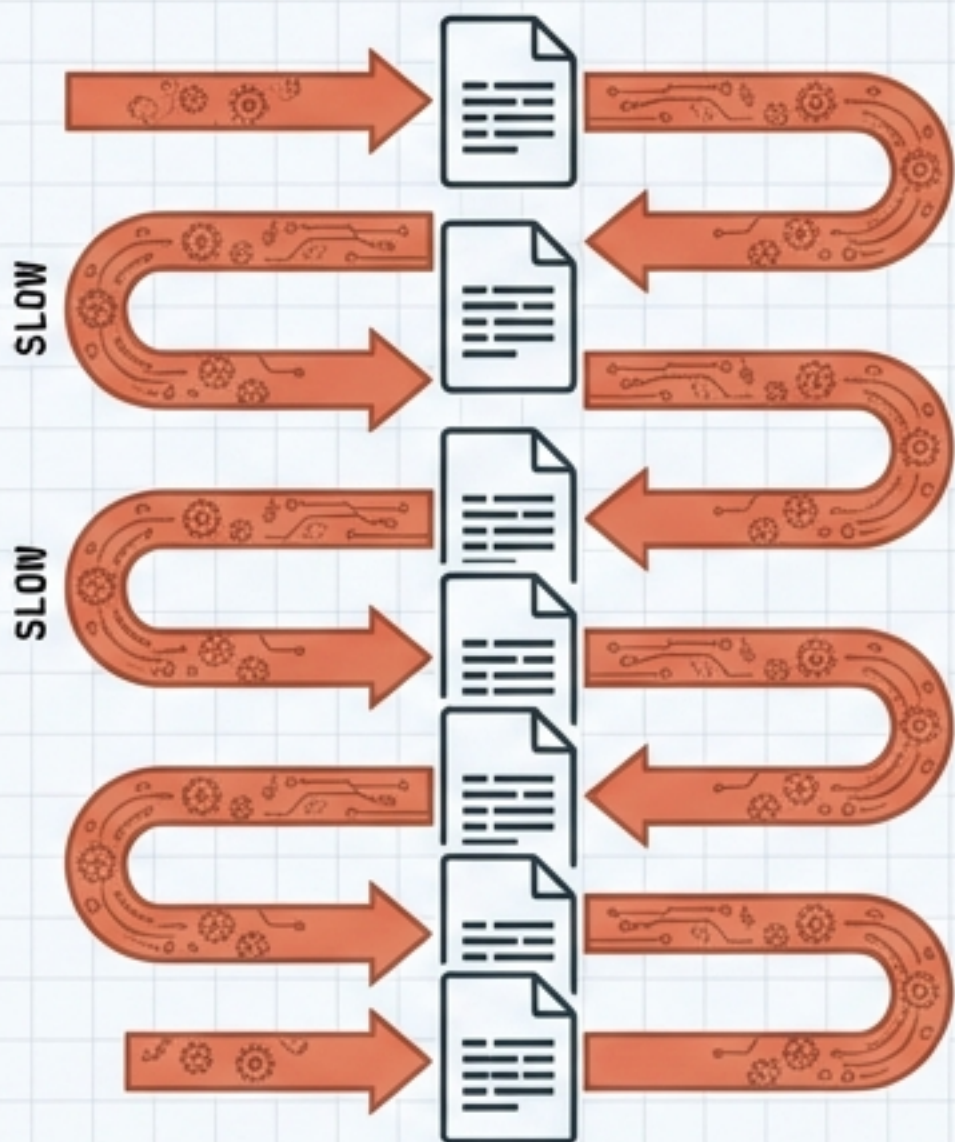
A Scalable, Graph-Based Indexing
Layer for Issue Tracking

VERSION: Draft v0.1
DATE: 01 Feb 2026
STATUS: Proposal for Review
AUTHOR: Architecture Planning Session



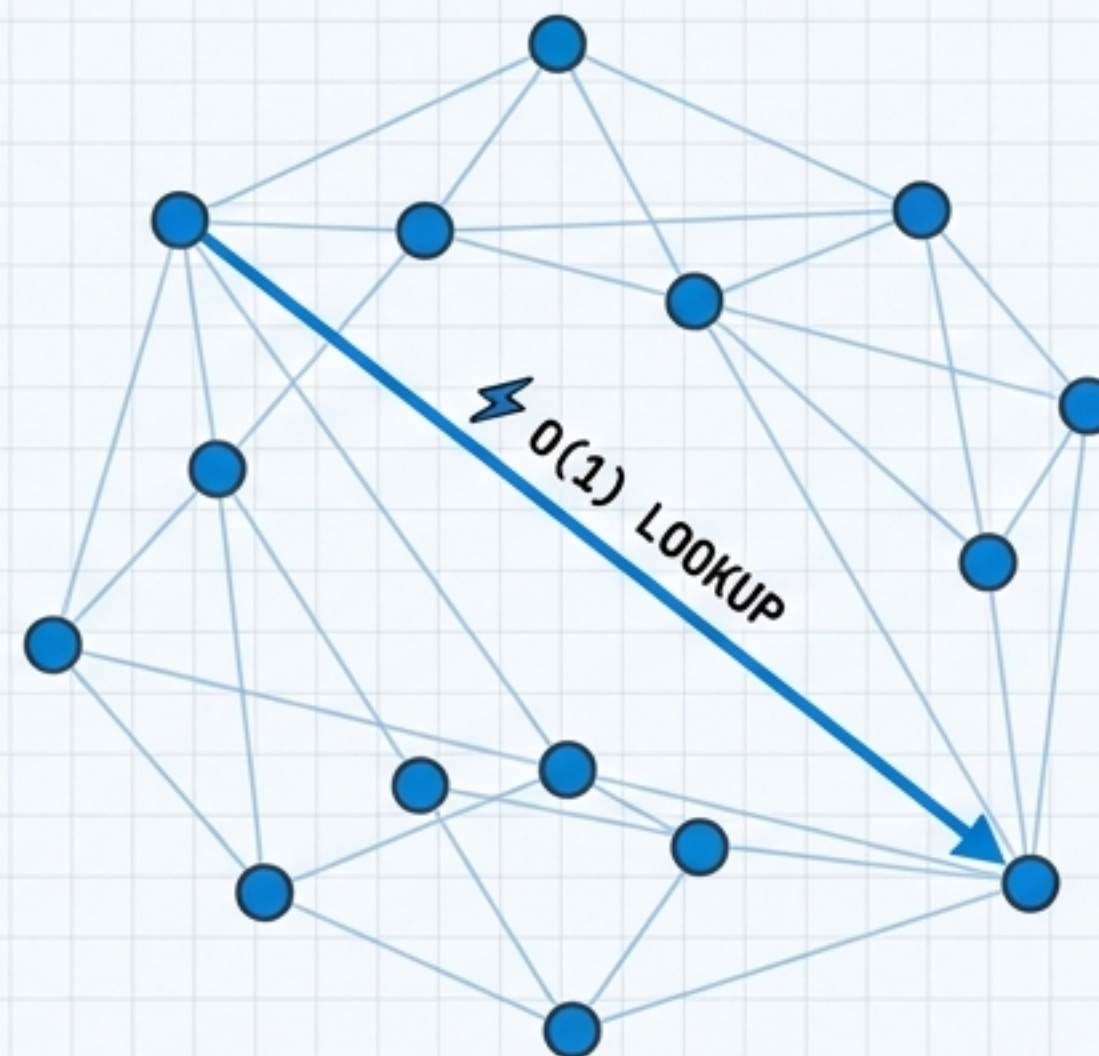
THE SCALABILITY BOTTLENECK: THE $O(N)$ PROBLEM

CURRENT STATE (FILE-BASED)



Limitation: To map relationships, the system must open every single file. $O(N)$ complexity degrades linearly as issue count grows.

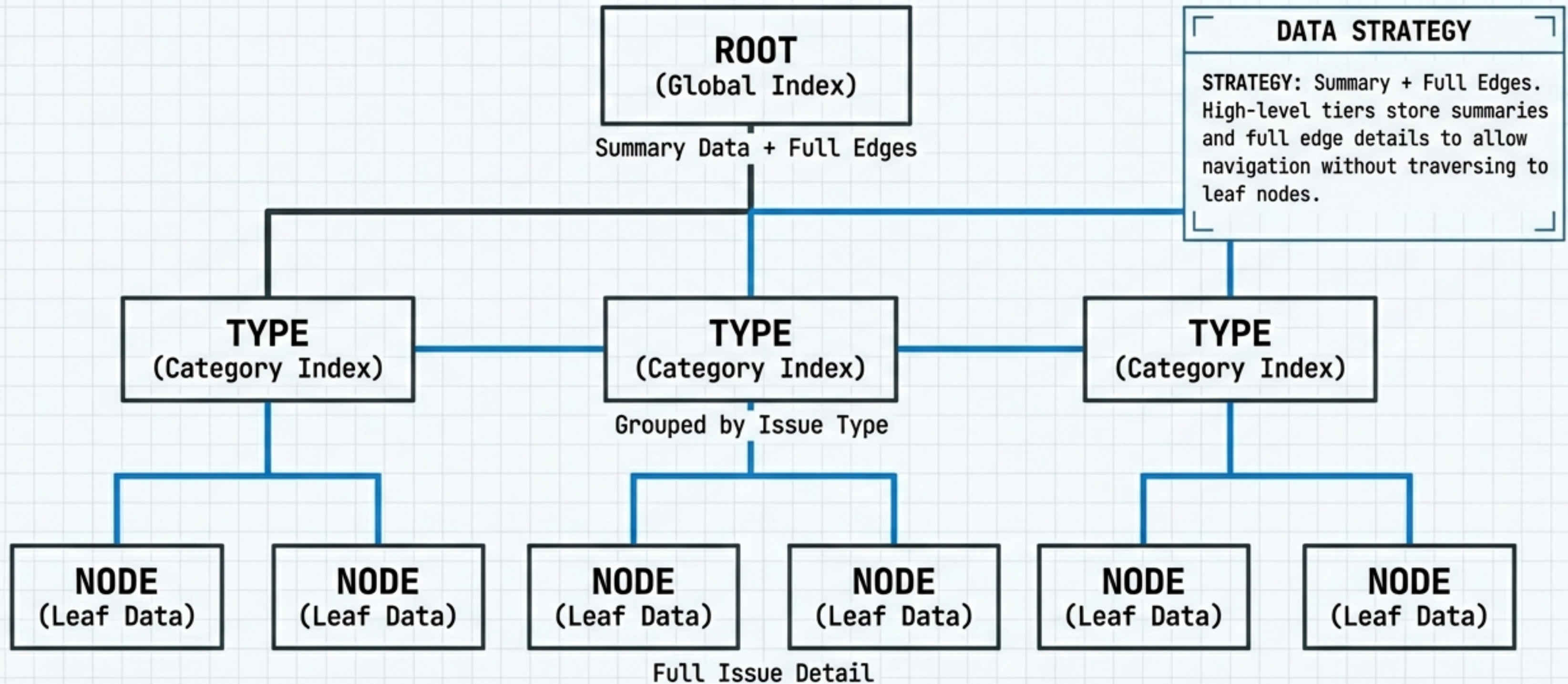
PROPOSED STATE (MGRAPH INDEX)



Solution: MGraph-DB acts as a hierarchical indexing layer. Enables $O(1)$ lookups while files remain the source of truth.

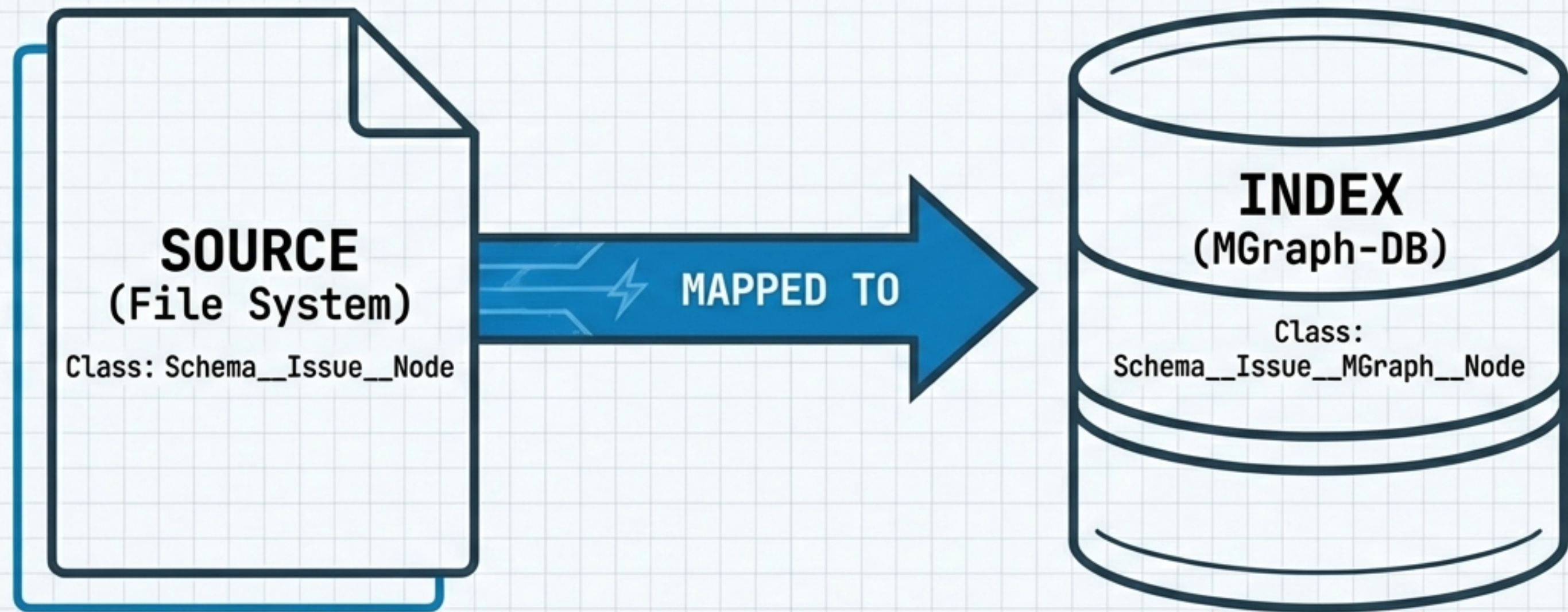
HIGH-LEVEL ARCHITECTURE: THE 3-TIER MODEL

Mirroring the file system hierarchy with MGraph-DB instances.



SCHEMA STRATEGY: NAMING & MAPPING

Strict naming conventions prevent collisions between the
File System and the Graph Index.

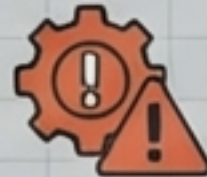


PREFIX RULE: All integration schemas must use 'Schema__Issue__MGraph__*' to clearly distinguish index data from source data.

IMPLEMENTATION: THE POWER OF TYPE_SAFE INTEGRATION

Raw Dictionary (Risky)

```
{  
  'id': 'issue_1',  
  'links': ['unkown_ref']  
  // No validation  
}
```



Type_Safe Class (Proposed)

```
@dataclass  
class Schema__Issue__MGraph__Node:  
    id: str  
    edges: List[Edge]  
  
    def validate(self):  
        # Runtime Validation  
        ...
```

BENEFITS OF TYPE SAFETY

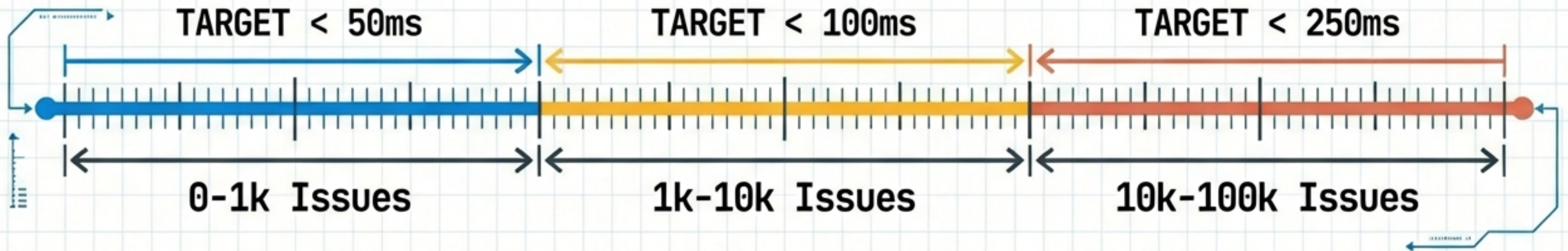
1. **Runtime Validation:** Catch bad data on assignment.
2. **Self-Documenting:** Explicit constraints.
3. **Consistency:** Uniform logic across system.

PERFORMANCE ARCHITECTURE: FROM O(N) TO O(1)

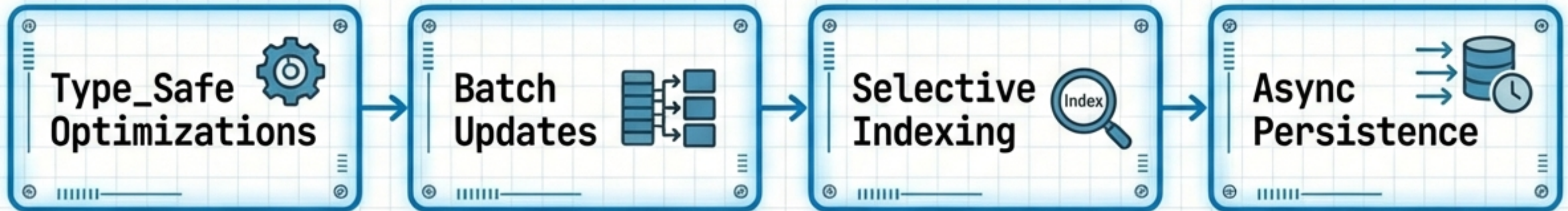
QUERY TYPE	BEFORE (File Scan)	AFTER (MGraph Index)
Find all links to a node	O(N) - Open all files	O(1) - Edge Index Lookup
Find nodes by status	O(N) - Open all files	O(1) - Value Index Lookup
Full Graph Visualization	O(N) - Load everything	O(1) - Pre-indexed
Single Node Detail	O(1) - Direct File Access	O(1) - Unchanged

PERFORMANCE THRESHOLDS & MONITORING

Proactive latency tracking to trigger optimizations.



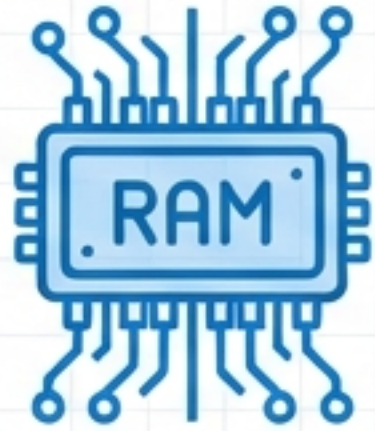
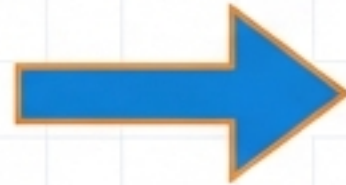
MITIGATION PROTOCOLS (Triggered on Warning)



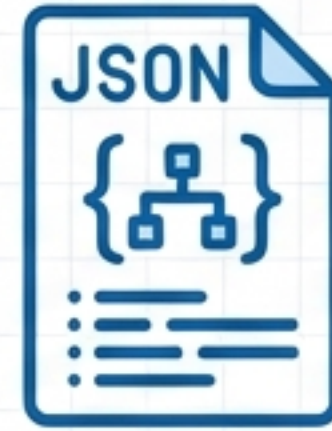
OPERATIONAL STRATEGY: SYNC & STORAGE



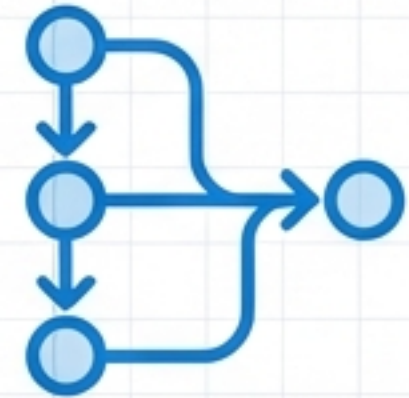
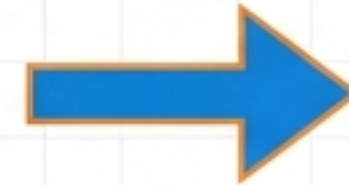
WRITE EVENT
(Single Writer)



UPDATE
IN-MEMORY INDEX



ASYNC PERSIST
(`_mgraph.db`)



GIT COMMIT

STORAGE

JSON Format.
Human-readable,
diffable, native.

GIT STRATEGY

Commit DB files.
Enables distributed
design & fast startup.

CONCURRENCY

Single Writer Model.
No complex locking
required.



KEY DESIGN DECISIONS & RATIONALE

WHY JSON STORAGE?



Native MGraph serialization, fully debuggable by humans, and works seamlessly with Git Diffs.

WHY COMMIT DB FILES?



Supports distributed, self-contained workflow. Allows instant startup $O(1)$ without re-indexing the whole repo.

WHY PERSISTENT INDEXES?



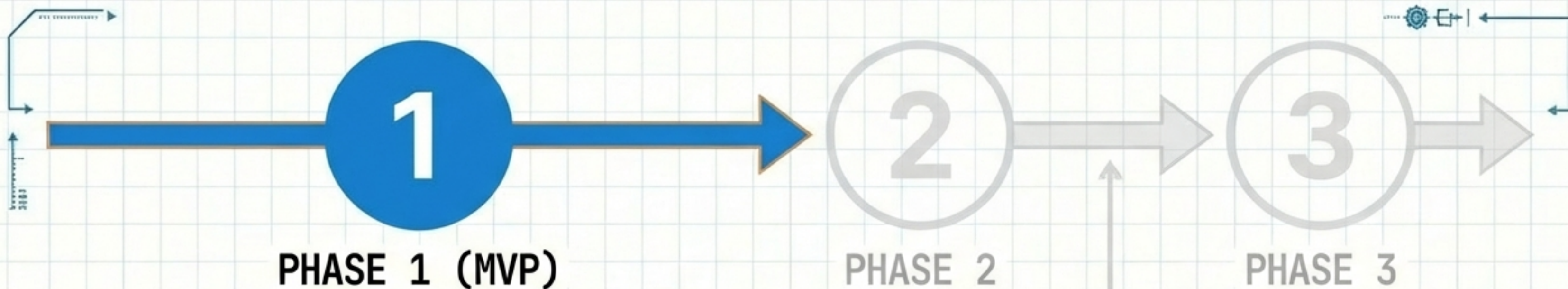
Designed for serverless environments (Lambda + S3). Loading a pre-built index is significantly faster than file scanning.

MIGRATION STRATEGY?



Rebuild on first access if '_mgraph.db' is missing. Graceful handling of legacy folders.

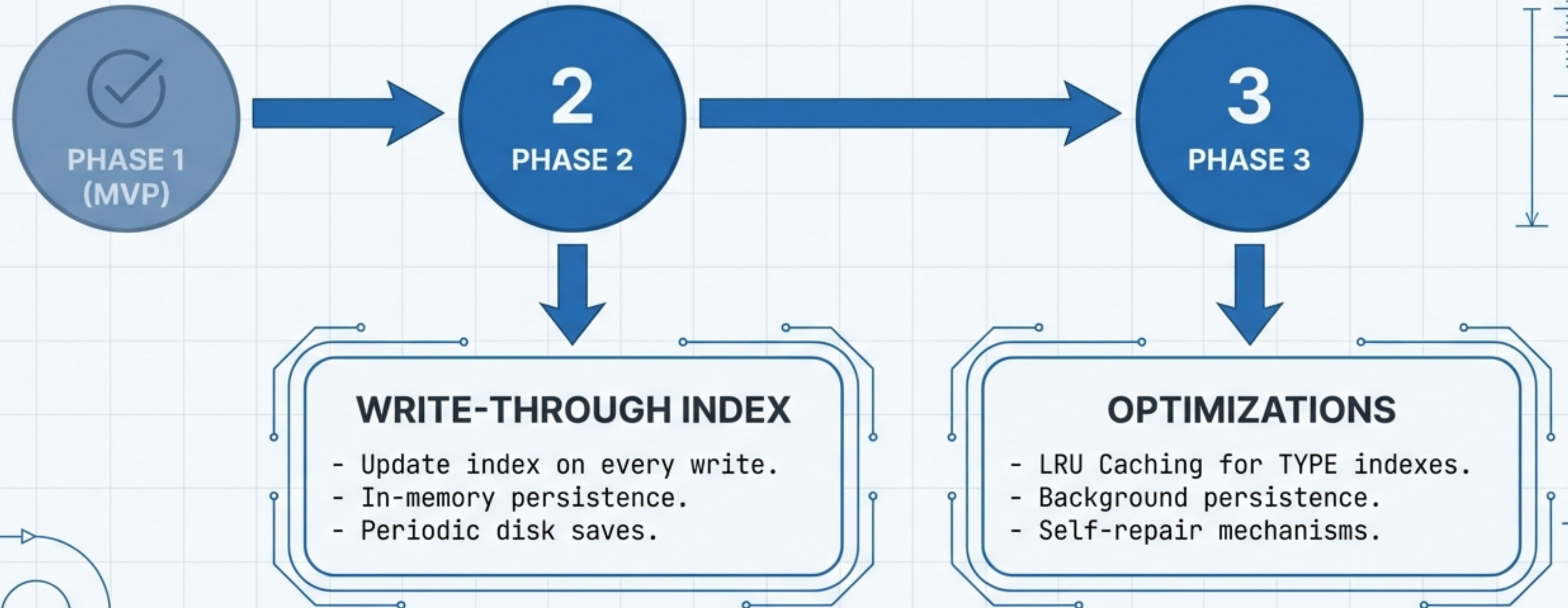
IMPLEMENTATION ROADMAP: PHASE 1 (MVP)



READ-ONLY INDEX

- Build index from files on startup.
- Query index for relationships.
- Fall back to files for details.
- No write-through (Rebuild on restart).

IMPLEMENTATION ROADMAP: PHASES 2 & 3



Inter Tight

VALIDATION: PLANNED EXPERIMENTS



EXP 01: ROOT DENSITY

JetBrains Mono

Test: Is 'summary only' data in ROOT sufficient?

Success: <50% of queries require fetching lower tiers.



EXP 02: EDGE DUPLICATION

JetBrains Mono

Test: Compare storing edges only in ROOT **vs.** duplicating in TYPE tiers.

Goal: **Balance** Storage Size **vs.** Query Speed.



EXP 03: DRIFT & REBUILD

JetBrains Mono

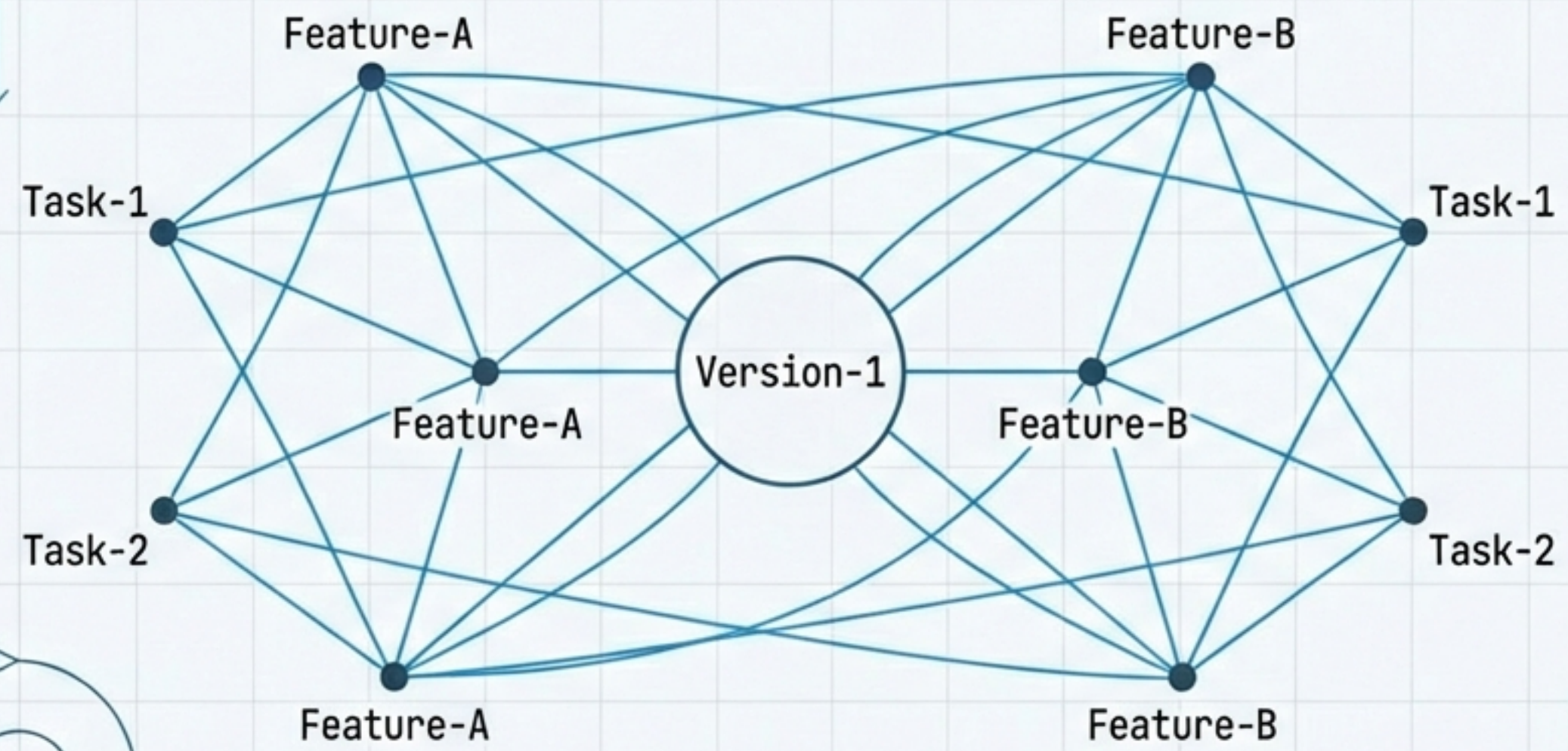
Test: Track index drift over **1,000 ops.**

Decision: Incremental updates **vs.** Full rebuilds on write.

Inter Tight

'FREE' BENEFITS: BEYOND JUST INDEXING

Unlocking advanced analysis through graph structure.



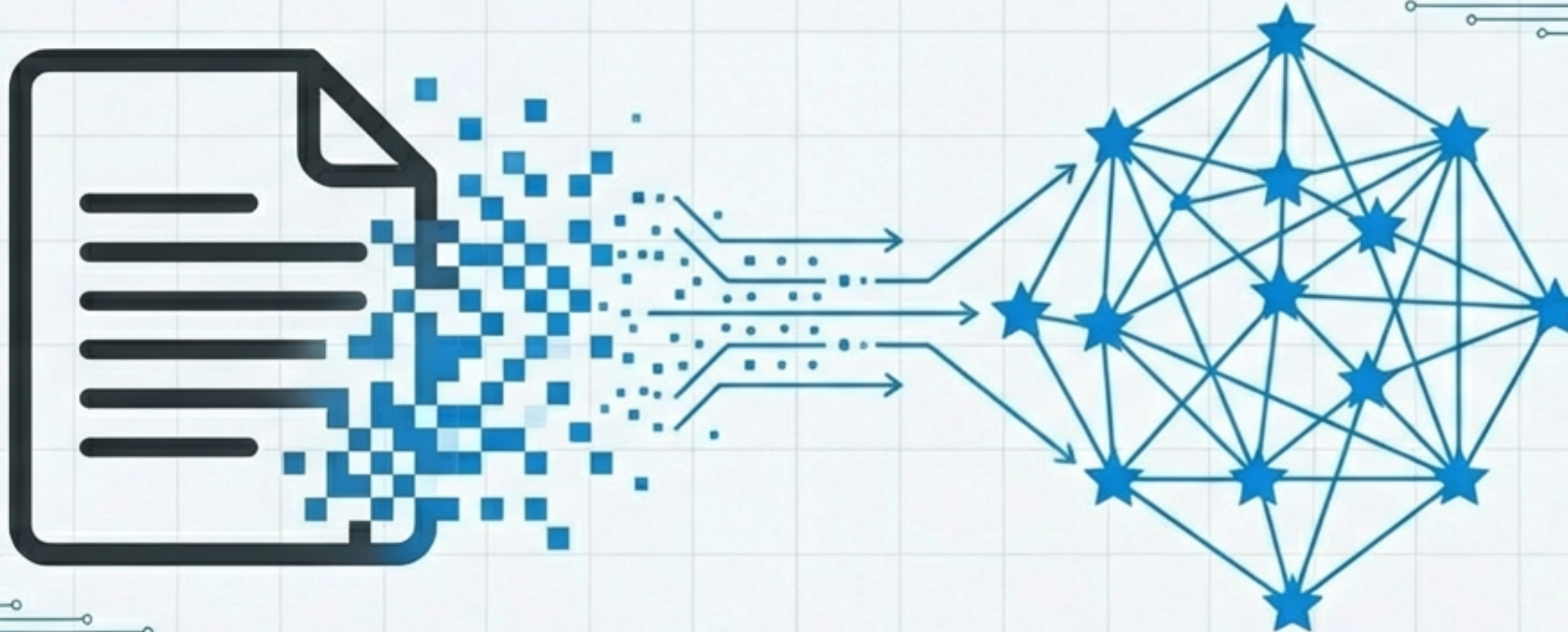
- 1. **VISUALIZATION:** Map versions to features instantly.
- 2. **FILTERING:** Advanced sub-graph queries.
- 3. **ANALYSIS:** Network density and bottleneck detection.

Inter Tight

NEXT STEPS & ACTION ITEMS

- ☐ **REVIEW ARCHITECTURE:** Confirm 3-Tier Model.
- ☐ **SCHEMA DEFINITION:** Define **Schema__Issue__MGraph__*** classes.
- ☐ **PROTOTYPE:** Build indexer & query engine.
- ☐ **EXPERIMENTATION:** Run Validation Tests (Exp 1-3).
- ☐ **INTEGRATION:** Connect to API to replace file scanning.
- ☐ **CONFLICT STRATEGY:** Await **Voice Memo #2**.

Inter Tight TRANSFORMING ISSUE TRACKING



MGraph-DB transforms our issue tracker from a static file repository into a queryable knowledge graph, achieving instant relationship lookups without sacrificing the simplicity of the file system.